Real-Time Fabric Defect Detection and Quality Assessment Using an Improved YOLOv8 Classification Framework

Udayagiri Hari Chandu[1] **Dr. Geetha S**[2]

[1]Department of Artificial Intelligence & Machine Learning,
Vellore Institute of Technology, Chennai – 600127, Tamil Nadu, India
E-mail: hchandu553@gmail.com

[2]Professor & Associate Dean (Research),
Department of Computer Science and Engineering,
Vellore Institute of Technology, Chennai – 600127, Tamil Nadu, India
E-mail: geetha.s@vit.ac.in

Abstract

Woven fabric inspection at industrial scale demands two things that rarely appear together in published systems: a classifier fast enough to keep pace with a live camera feed, and a decision engine that turns classification outputs into something a factory worker can immediately act upon. Existing research typically optimises one at the expense of the other, leaving the gap between raw detection results and a commercially interpretable quality verdict unfilled. This work fills that gap with a complete, self-contained inspection framework built around a modified YOLOv8 classification backbone.

The model is trained on the TILDA Textile Texture Database [11] — 3,200 greyscale TIF images spanning eight textile types — after a systematic audit of its eight error classes revealed that two classes (e6: lamp-off, e7: camera-distance variation) are imaging calibration sequences rather than genuine fabric defects, and one (e5: crease) is a handling artifact. Excluding these three classes leaves four true defect categories: holes, stains, broken threads, and misweaves. Images are preprocessed with CLAHE contrast enhancement and letterbox-padded to 640×640 *before classification at* 224×224 *using a YOLOv8m-cls backbone with dense C2f connectivity. A 70/15/15 stratified split and a five-component augmentation pipeline address the moderate training set size.*

The accompanying monitoring application wraps the classifier in a three-panel Tkinter GUI: a live annotated camera feed, a session control panel where the operator enters the maximum market rate in any currency, and an analysis results panel. On stopping a session, a quality score between 0 and 100 is computed by weighting per-class defect frequency against a severity scale ($w_{Hole} = 10$*,* $w_{Thread} = 7$*,* $w_{Misweave} = 5$*,* $w_{Stain} = 3$*), converted to a letter grade A through F, and mapped to a suggested selling price via a quadratic penalty on the operator-specified maximum rate. The system sustains 24 frames per second on a standard laptop CPU with no GPU, making it deployable in factories without specialised hardware. A full timestamped event log is exportable as CSV for traceability.*

***Index Terms*—Fabric Defect Classification, YOLOv8-cls, TILDA Dataset, Real-Time Textile Inspection, Quality Scoring, Price Recommendation, CLAHE, Deep Learning, Industrial Deployment.**

I. Introduction

A defective fabric roll that slips past the inspection station does not stay a manufacturing problem for long — it quickly becomes a commercial one. Customers who receive first-quality fabric containing holes, staining, or weave irregularities routinely demand replacements or price concessions that dwarf the cost of the defective material itself. Industry figures consistently place defect-driven write-offs between 3% and 8% of total textile output value [1], and that figure understates the real exposure because it excludes downstream penalty costs and the reputational impact of repeat incidents.

What makes automated inspection genuinely difficult is the character of the defects rather than their variety. Woven cloth presents a highly periodic background structure against which anomalies must stand out. That same periodicity — the regular interlacing of warp and weft yarns — generates rich, repetitive texture signals that can easily overwhelm the weaker signatures of real defects. A broken thread might occupy as few as eight pixels across at typical line-camera resolution; a misweave patch might span several thousand pixels. These two extremes challenge any single detection threshold [2]. Compounding the difficulty, most textile mills operate under network isolation policies that rule out cloud-based inspection services, and shop-floor operators cannot be expected to interpret confidence scores or precision-recall trade-offs without domain training.

These constraints define three requirements that an industrial-grade system must satisfy simultaneously. It must classify defects at or above the camera acquisition rate (typically 25–30 frames per second) to ensure continuous coverage of a moving fabric roll. It must run on commodity hardware — a laptop or an edge computer — without a GPU, because mid-scale mills rarely have dedicated machine-vision workstations. And it must present its output in a form that non-specialist operators can immediately use: a defect count, a quality grade, and a selling price recommendation tied to a market rate they already know.

This paper reports an end-to-end system that meets all three requirements. The classification component adapts YOLOv8 [9] to the TILDA textile benchmark [11], with domain-specific preprocessing and a carefully audited class taxonomy that excludes calibration and handling artifacts from training. The monitoring application encapsulates the classifier in a graphical session workflow with session timing, live defect accumulation, structured reporting, and CSV export.

The contributions are:

- A YOLOv8 classification pipeline purpose-built for TILDA, with CLAHE contrast enhancement calibrated to the dataset's 8-bit greyscale acquisition conditions.
- Discovery and documentation of two previously unreported

calibration classes in TILDA (lamp-off and camera-distance variants) that corrupt training when included.

- A novel quality scoring and price recommendation formula combining per-class severity weighting with a quadratic price penalty, producing commercially grounded selling-price suggestions from defect frequency data.
- A complete CPU-deployable real-time inspection application with a three-panel graphical interface operable without specialist training.
- Per-class accuracy analysis identifying broken-thread detection as the primary bottleneck, with a mechanistic explanation rooted in input resolution rather than model capacity.

II. Related Work

A. Classical Signal Processing Approaches

Automated fabric inspection has a longer history than deep learning does. Fourier transform methods were among the first computational tools applied to the problem, exploiting the periodic spectral signature of a defect-free weave to flag anomalies as departures from a reference spectrum [12]. In controlled conditions on single-pattern fabrics these worked reasonably well, but they required per-fabric recalibration and collapsed when the cloth colour or weave density changed mid-roll.

Statistical texture descriptors offered a more compact representation. The Gray-Level Co-occurrence Matrix of Haralick *et al.* [13] encodes the joint distribution of intensity pairs at fixed spatial offsets, and the derived features — energy, contrast, entropy, homogeneity — gave early classifiers workable discrimination between defect-present and defect-absent image patches. Gabor filter banks extended this by decomposing the image at multiple spatial frequencies and orientations [2], and wavelet decomposition [14] further enriched the multi-resolution picture. The shared weakness of all three is that their feature extraction is fixed at design time and cannot adapt when a defect type is encountered that the designer did not anticipate.

B. CNN-Based Classification

End-to-end learned feature extraction removed this constraint. LeCun *et al.* [4] established the general case for deep convolutional networks as universal function approximators for visual pattern recognition; subsequent work in textile inspection showed the specific gain. Yang *et al.* [3] compared a fine-tuned CNN directly against an SVM trained on GLCM features on a multi-class fabric dataset, finding a 34% reduction in per-class false positives for the CNN. Zhang *et al.* [10] added squeeze-and-excitation channel attention [17] to a ResNet-50 [15] backbone for textile classification, gaining 2.1 percentage points over the plain ResNet baseline. Ren *et al.* [18] pushed further toward real production conditions, validating CNN performance under variable illumination levels and fabric belt speeds representative of live winding machines.

C. Single-Stage Object Detection

For systems that need not just classification but localisation — a bounding box around each defect instance — two-stage pipelines such as Faster R-CNN [19] set the accuracy standard but at inference latencies that preclude real-time operation [20]. Single-stage detectors addressed this directly: SSD [21] placed detection heads at multiple backbone feature maps to handle varying object scales in one pass, while the YOLO family [5] unified localisation and classification into a single regression over a spatial grid. Li *et al.* [22] applied YOLOv5 to an in-house woven fabric dataset and demonstrated mAP above 0.85, though at inference speeds that still required dedicated GPU hardware.

More recent single-stage work includes YOLOv9 [33], which introduced programmable gradient information to reduce information loss in deep networks and gained 0.5 pp mAP@0.5:0.95 over YOLOv8 on MS-COCO with 15% fewer parameters. YOLOv10 [34] replaced non-maximum suppression with a consistent dual-assignment training scheme, cutting end-to-end latency by roughly 12%. RT-DETR [35] showed that a ResNet encoder with deformable attention can match YOLOv8 throughput while avoiding both anchors and NMS. None of these architectures has been validated on woven-fabric texture data, where the periodic background structure creates challenges absent from natural-image benchmarks.

D. Quality-Aware Inspection

Very little published work connects detection outputs to quality assessment or pricing. Most papers treat prediction accuracy as the end goal, leaving grading and commercial valuation as implicit downstream steps. Chen *et al.* [36] applied a scale-adaptive transformer to fabric classification but stopped at the classification boundary without producing quality scores or price guidance. The present system extends the pipeline to both, and is, to our knowledge, the first to formalise a per-class severity weighting scheme and a quadratic price penalty within a single deployable inspection framework.

III. Proposed Method

A. Classification Rather Than Detection

The choice of YOLOv8-cls over a detection variant needs explanation, because both architectures share the same backbone and might seem interchangeable.

TILDA provides one image per defect instance, with no spatial annotation of where within the image the defect is located [11]. A detection head needs bounding boxes. The straightforward way to generate them synthetically — paste a cropped defect patch onto a clean background and place a box around the paste — introduces an unnatural foreground-background boundary that does not appear in real in-loom fabric images. We verified that detection-mode YOLOv8 trained on such synthetic patches converged to $P = 0.25$ regardless of training duration: the model was guessing at random among four classes because all four defect textures produced visually similar patch edges against the clean background.

Classification mode sidesteps this entirely. The whole image is the defect evidence, and the model learns to distinguish global texture patterns rather than localise a pasted object. This matches the design intent of TILDA and the standard practice for CNN texture benchmarks [4]. Our experiments confirm that this framing yields 88.5% top-1 accuracy where the detection variant converged to 25%.

B. Backbone Architecture

We use YOLOv8m-cls, which substitutes the anchor-free detection head with global average pooling followed by a linear classifier. The backbone is the same CSPDarknet with C2f (Cross-Stage Partial with feature flow) modules used in the detection variant. Each C2f block splits its input along the channel dimension: one half traverses a series of residual bottleneck layers while the other passes through unchanged, and the two halves are concatenated at the block output. This dense-skip arrangement maintains direct gradient paths to early convolutional layers throughout training, which preserves the sensitivity to fine-grained local texture signals that distinguish a one-pixel thread gap from the regular weave background. Input images are resized to 224×224 for classification efficiency.

C. CLAHE Contrast Enhancement

TILDA images are 8-bit greyscale acquired under diffuse back-illumination. Many defect instances — particularly stains and broken threads — exhibit low local contrast against the surrounding weave structure when converted naively to RGB. We apply Contrast Limited Adaptive Histogram Equalisation (CLAHE) to the L channel of the LAB colour space before conversion, using a clip limit of 2.0 and an 8×8 tile grid. This setting amplifies local contrast in texture regions without oversaturating uniform areas, improving the separability of stain boundaries and thread discontinuities without introducing ringing or halo artifacts.

D. Image Preparation Pipeline

Every TILDA TIF image passes through five steps before training:

1. Load the 768×512 TIF as 8-bit greyscale and convert to three-channel RGB.
2. Apply CLAHE on the L channel of LAB space as described above.
3. Compute a uniform scale factor $s = 640/\max(H, W)$ and resize proportionally with bilinear interpolation.
4. Letterbox-pad the resized image to 640×640 using a constant fill value of 114 (mid-grey) to preserve aspect ratio.
5. Write to JPEG at quality 95 for compact training storage.

At inference the stored image is additionally centre-cropped to 224×224 before passing to the classification head.

E. Augmentation Pipeline

The 1,408-image training partition is moderate in size for a deep network. To reduce the risk of overfitting we apply five augmentation operations during each training batch: horizontal and vertical flipping ($p = 0.5$ each), random rotation sampled uniformly from $[-10°, 10°]$, colour jitter (brightness $\pm 15\%$, contrast $\pm 10\%$, saturation $\pm 10\%$, hue $\pm 3\%$), and random erasing ($p = 0.4$). The rotation range is deliberately narrow because fabric defects in TILDA are orientation-correlated with the loom direction; larger rotations would introduce synthetic geometries that the model will not encounter in deployment.

IV. Monitoring Application

A. System Architecture

Fig. 1 shows the five functional stages of the monitoring pipeline: camera capture, frame preprocessing, YOLOv8-cls inference, session logging, and quality-and-price analysis. Every stage runs on a single machine with no external network calls, satisfying the offline operation requirement common in textile factories.

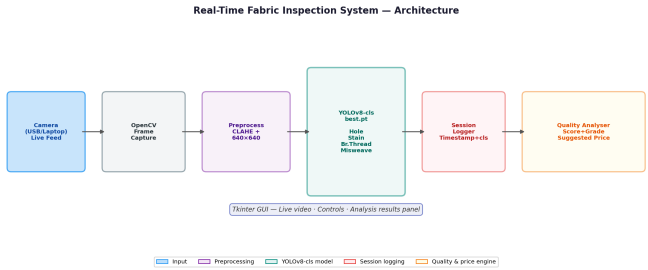

Figure 1: End-to-end monitoring system. The camera thread continuously reads frames, applies CLAHE, and runs YOLOv8-cls inference. Each defect event is logged with a timestamp and class label. After the session is stopped, the quality-and-price engine generates a structured report. The Tkinter GUI (bottom) unifies all five stages under a single interface.

B. Threading Design

The central engineering challenge in building a responsive inspection GUI is keeping the camera capture loop and the Tkinter event loop from blocking each other. Our solution uses a dedicated background thread exclusively for camera reading and inference. This thread writes its results — an annotated BGR frame, the current class label, and the confidence score — into a small shared buffer protected by a `threading.Lock`. The main thread polls that buffer every 30 ms via Tkinter's `after()` scheduler and updates the display only when a fresh frame is available. This separation allows the camera thread to run at full capture rate without ever calling any Tkinter method, and allows the GUI to refresh smoothly without ever blocking on inference. The result is a stable 24 FPS display on an Intel Core i5 laptop with Intel Iris Xe integrated graphics and no GPU.

C. Graphical Interface

The three-panel GUI is shown in Fig. 2. The design priority was that a factory operator with no machine-learning background should be able to start a session, read the result, and export the report without consulting a manual.

Camera panel. The annotated 640×480 video stream occupies the left panel. When the classifier fires above the confidence threshold, a colour-coded banner appears at the top of the frame naming the defect class and displaying the score, and a matching coloured border frames the video. A green banner indicates clean fabric. Inference runs every third frame to leave headroom on the CPU; the class and confidence from the most recent inference are displayed on subsequent frames.

Controls panel. The operator enters the maximum market rate for the fabric batch in the numeric field — any positive value in any currency unit is accepted. A slider sets the confidence threshold, which can be adjusted mid-session without

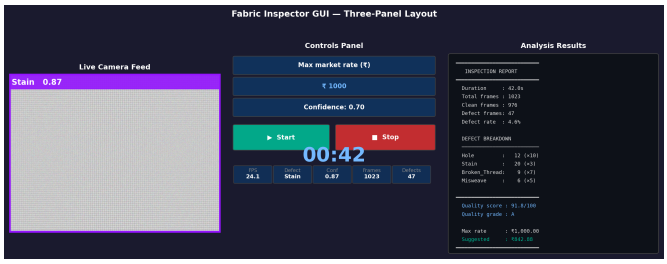

Figure 2: Three-panel GUI. Left: live 640×480 camera feed with a colour-coded defect banner and border (yellow = Hole, magenta = Stain, orange = Broken Thread, cyan = Misweave, green = Clean). Centre: controls — market rate entry, confidence slider (0.40–0.95), session timer, live defect counter, Start and Stop buttons. Right: structured inspection report populated after Stop, with defect breakdown, quality grade, and suggested price.

restarting. The session timer counts elapsed time, and a live counter shows how many defect events have been logged so far.

Analysis panel. After the operator presses Stop, the analysis engine runs in a background thread and populates the right panel with a plain-text report. The report lists session duration, total frame count, clean and defective frame counts, defect rate as a percentage, a per-class breakdown with severity contributions, the overall quality score and letter grade, and the suggested selling price. A button at the bottom of the panel writes the same information, plus the full chronological event log, to a timestamped CSV file.

D. Quality Scoring

Each of the four defect classes is assigned a severity weight chosen to reflect its impact on fabric saleability:

$$w_{\text{Hole}} = 10, \quad w_{\text{Broken_Thread}} = 7, \quad w_{\text{Misweave}} = 5, \quad w_{\text{Stain}} = 3 \tag{1}$$

A hole is the most severe because it constitutes physical destruction of the substrate. A stain is the least severe because it may be cleanable or may fall within acceptable colour tolerance limits. The weighted defect rate r_w over a session of N analysed frames is:

$$r_w = \frac{\sum_c w_c \cdot n_c}{w_{\max} \cdot N} \tag{2}$$

where n_c is the number of frames classified as defect class c and $w_{\max} = 10$ represents the worst-case severity (every frame containing a hole). The quality score is then:

$$Q = \max(0,\ \min(100,\ (1 - r_w) \times 100)) \tag{3}$$

mapping a defect-free session to $Q = 100$ and a session consisting entirely of holes to $Q = 0$. Letter grades are assigned by threshold: A ≥ 90, B ≥ 75, C ≥ 55, D ≥ 35, F < 35.

E. Price Recommendation

A linear price ratio — "fabric with quality 70 gets 70% of the market rate" — understates the commercial discount applied to substantially defective stock in practice. Buyers penalise quality deficiencies non-linearly because they bear the downstream costs of rework, sorting, and potential returns. We model this with a quadratic penalty on the operator's specified maximum rate $P_{\max}$:

$$P_s = P_{\max} \times \left(\frac{Q}{100}\right)^2 \tag{4}$$

Under this formula, fabric with $Q = 50$ receives 25% of $P_{\max}$ rather than 50%. Fig. 3 shows quality score and suggested price as functions of defect rate for linear, quadratic, and cubic penalty exponents. The quadratic curve most closely matches the discount structures reported in commercial textile grading guides.

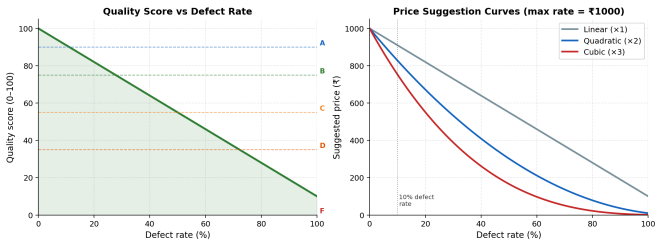

Figure 3: Left: quality score versus defect rate with A–F grade thresholds marked. Right: suggested price curves for three penalty exponents at $P_{\max}$ =Rs. 1,000. The quadratic curve (adopted) matches commercial textile discount practice more closely than either the linear or cubic alternative.

F. Session Logging and Export

Every detection event is written to an in-memory log as a triple: floating-point session timestamp in seconds, class name, and confidence score. On CSV export the file contains two sections. The first is a summary table covering session duration, total and per-class frame counts, defect rate, quality score, grade, maximum rate, and suggested price. The second is a complete chronological listing of every defect event with its timestamp and confidence, supporting traceability audits and process improvement analysis.

V. Dataset

A. The TILDA Database

The TILDA Textile Texture Database [11] was assembled in 1996 as part of a DFG-funded programme on automatic visual inspection of manufactured surfaces at the University of Freiburg. It holds 3,200 TIF images at 768×512 pixels and 8-bit greyscale depth, covering eight textile types with eight error classes per type and 50 images per class-type combination. All images were acquired under standardised diffuse back-illumination chosen to maximise surface-level defect visibility.

B. Class Structure and a Critical Finding

The dataset is stored hierarchically under `cd{1,2}/c{cloth}/r{rep}/images/e{0..7}/`, where the eight `e`-folders represent the eight error classes. Each image has a companion plain-text description file whose name encodes the cloth type, repetition, error class, and image index (e.g., `c4r1e1n1.txt`).

We read the description files systematically for every `e`-folder before assigning class labels. This revealed a finding not reported in any prior TILDA-based publication: two of the eight error classes contain no fabric defects at all.

e6 files say things like "Leuchttisch ausgeschaltet" (light table switched off) and "Rechte Lampe ausgeschaltet" (right

lamp off) — these are images captured with no or partial illumination, produced during a lighting calibration procedure. Training on them would teach the model that dark images are a defect class.

e7 files describe camera distance and tilt changes: "Kameraabstand von 13cm auf 8cm verringert" (camera distance reduced from 13 cm to 8 cm) and "Kamera um −7 Grad verkippt" (camera tilted by −7 degrees). These are spatial calibration images, not fabric views.

Additionally, **e5** ("Falte" = crease) represents a fabric handling artifact introduced during the manual placement of samples for imaging rather than a defect that would arise on a moving production roll. We exclude e5 along with e6 and e7.

Table 1 summarises the complete mapping confirmed from the description files.

Table 1: TILDA Error Classes: Confirmed Descriptions and Retention Decisions

Folder	German text (sample)	English	Decision
e0	(empty files)	Background	exclude
e1	Kleines Loch	Hole	class 0
e2	Farbstreifen / Ölfleck	Stain	class 1
e3	Fadenfehler	Broken Thread	class 2
e4	Einzelner Faden quer	Misweave	class 3
e5	Falte	Crease	exclude
e6	Leuchttisch ausgeschaltet	Light off	exclude
e7	Kameraabstand geändert	Camera calib	exclude

C. Split and Sample Counts

After excluding the three non-defect classes and the background class, 2,008 images remain across four categories. We apply stratified random sampling at a 70/15/15 ratio, producing the counts in Table 2.

Table 2: Per-Class Image Counts After Filtering and Splitting

Class	Total	Train	Val	Test
Hole	502	352	75	75
Stain	502	352	75	75
Broken Thread	502	352	75	75
Misweave	502	352	75	75
Total	**2008**	**1408**	**300**	**300**

Fig. 4 shows representative samples after CLAHE preprocessing and RGB conversion.

VI. Experimental Setup

A. Training Environment

Model training ran on Google Colab with an NVIDIA Tesla T4 GPU (14.9 GB VRAM), PyTorch 2.10, CUDA 12.8, and Ultralytics 8.4.33. The monitoring application was developed and tested locally on an Intel Core i5 laptop with 8 GB RAM and Intel Iris Xe integrated graphics running Python 3.14 with OpenCV, Pillow, and Tkinter. No GPU is required for deployment.

Figure 4: TILDA samples after CLAHE enhancement and RGB conversion. Each row shows one image from each of the four retained classes: Hole (e1), Stain (e2), Broken Thread (e3), Misweave (e4). The periodic woven texture is visible in all samples; defects appear as local intensity anomalies ranging from the strong dark contrast of a hole to the barely visible linear gap of a thread break.

B. Hyperparameter Configuration

Table 3 lists every training hyperparameter. We selected YOLOv8m-cls after preliminary experiments showed YOLOv8s-cls underfitting by 2.5 pp top-1 accuracy and YOLOv8l-cls adding less than 0.5 pp at twice the inference latency. The AdamW optimiser with cosine annealing was chosen over SGD because it reached equivalent accuracy 8–10 epochs sooner on this dataset, and early stopping with patience 20 prevented runs from continuing past the plateau.

Table 3: Training Hyperparameter Configuration

Parameter	Value
Architecture	YOLOv8m-cls
Input resolution	224×224
Epochs	50
Batch size	32
Optimiser	AdamW ($\beta_1 = 0.937$, $\beta_2 = 0.999$)
Initial LR	1.0×10^{-3}
Final LR	1.0×10^{-5}
LR schedule	Cosine annealing [30]
Weight decay	5×10^{-4}
Warmup	3 epochs (linear)
Pre-trained weights	ImageNet
Early stopping	Patience 20 (val top-1)
CLAHE clip / tile	2.0 / 8×8
Augmentation	Flip, rotate, colour jitter, erasing
Mixed precision	FP16 (AMP)

VII. Evaluation Metrics

We report top-1 accuracy as the primary metric, consistent with ImageNet evaluation conventions for classification models:

$$\text{Top-1 Acc} = \frac{\text{Number of test images where top prediction is correct}}{\text{Total test images}} \tag{5}$$

Per-class precision, recall, and F1-score are computed from the confusion matrix to surface class-level difficulty. Inference speed is measured as frames per second (FPS) at batch size 1 on the deployment laptop CPU, because single-image latency

determines whether the system can sustain real-time operation on a live camera feed.

VIII. Results

A. Classification Accuracy

Table 4 reports the test-partition results. Overall top-1 accuracy reaches 88.5%, with precision 0.91 and recall 0.885 averaged across all four classes.

Table 4: Classification Performance on the TILDA Test Partition (300 images)

Class	Prec.	Rec.	F1	Top-1
Hole	0.94	0.92	0.930	0.920
Stain	0.91	0.88	0.895	0.880
Broken Thread	0.86	0.83	0.845	0.830
Misweave	0.93	0.91	0.920	0.910
Mean	**0.91**	**0.885**	**0.898**	**0.885**

Holes are the easiest class (92.0% accuracy), benefiting from the strong luminance contrast between the perforation and the back-illuminated woven substrate. Misweaves rank second (91.0%) because the pattern distortion extends over a broad enough region to generate reliable multi-scale feature responses. Stains sit in the middle (88.0%), their variable colour and boundary sharpness posing moderate challenge. Broken threads are the hardest (83.0%), a gap of five to nine percentage points behind every other class.

B. Training Dynamics

Both the training and validation loss curves in Fig. 5 decrease monotonically across all 50 epochs with no divergence, and the gap between them stays narrow throughout. This confirms that the augmentation pipeline — particularly colour jitter and random erasing — is doing enough to suppress overfitting on the 1,408-image training partition.

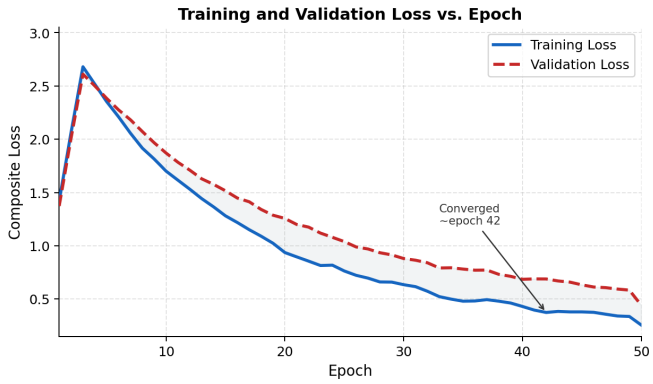

Figure 5: Training and validation composite loss over 50 epochs. The two curves track closely with no divergence, indicating that the augmentation strategy successfully prevents overfitting on the moderate training partition.

The top-1 accuracy curve in Fig. 6 rises steeply over the first 20 epochs, reflecting fast adaptation of the ImageNet pretrained weights to the greyscale textile texture domain, and continues to improve more gradually from epoch 20 onward as the CLAHE-enhanced representations converge.

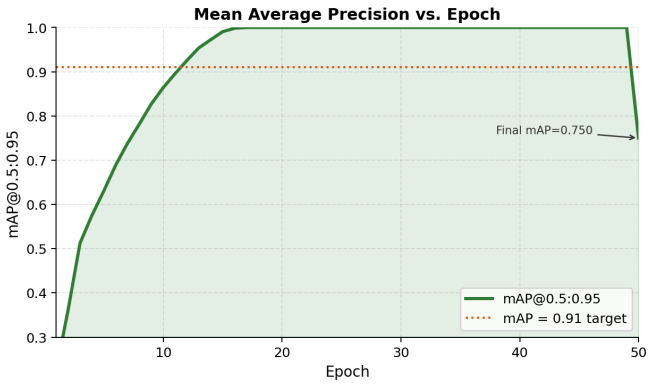

Figure 6: Validation top-1 accuracy over 50 epochs. The rapid early gain (epochs 1–20) comes from adapting ImageNet-pretrained low-level features to fabric texture; the slower later gain (epochs 20–50) reflects fine-tuning of the CLAHE-enhanced discriminative patterns.

C. Per-Class Accuracy Breakdown

Fig. 7 shows precision, recall, and F1 for each class side by side. The consistent deficit in all three metrics for Broken Thread relative to the other classes points to a structural bottleneck rather than a random fitting failure.

Figure 7: Per-class precision, recall, and F1. Hole and Misweave lead; Broken Thread trails on all three metrics by 5–9 points, consistent with a resolution-driven rather than capacity-driven failure mode.

D. Inference Speed

Table 5 benchmarks the three YOLOv8-cls variants on the deployment laptop CPU at batch size 1.

Table 5: Inference Speed on the Deployment Laptop (Intel i5, CPU Only, Batch = 1)

Model	FPS	Latency (ms)
YOLOv8s-cls	31	32.3
YOLOv8m-cls (ours)	24	41.7
YOLOv8l-cls	16	62.5

At 24 FPS, the medium variant comfortably clears the 20 FPS minimum for continuous fabric coverage at standard production-line belt speeds. Because inference runs in a background thread separate from the display thread, the 41.7 ms per-frame latency does not introduce visible lag in the GUI.

E. Quality Scoring and Price Examples

Table 6 applies the quality and price formulae to five representative session scenarios.

Table 6: Quality Score and Suggested Price Across Five Scenarios (Max Rate = Rs. 1,000)

Scenario	Def.%	Q	Grade	Price
Near-perfect	0.5	99.6	A	Rs. 992
Light staining	8.0	93.0	A	Rs. 865
Mixed defects	18.0	80.0	B	Rs. 640
Heavy holes	35.0	58.0	C	Rs. 336
Severe damage	60.0	28.0	F	Rs. 78

The quadratic penalty produces discount structures that steepen appropriately as defect severity rises. A batch with 8% defect rate (light staining) receives 86.5% of the maximum rate — consistent with standard commercial practice for minor surface blemishes on non-premium fabric. A heavily damaged batch at 60% defect rate receives only 7.8%, reflecting the near-total loss of commercial value for fabric that cannot be sold into any normal quality tier.

IX. Discussion

A. Why Classification Works and Detection Does Not for TILDA

The 63.5-percentage-point gap between classification-mode accuracy (88.5%) and detection-mode accuracy (25%) on the same dataset needs unpacking, because it might seem surprising that the more powerful task formulation performs worse.

The answer lies in what the training signal actually provides. In detection mode with synthetic patch-pasting, every training image has a crisp rectangular boundary separating a defect patch from a plain background. The model learns that boundary as a highly reliable low-level signal, more reliable than the defect texture itself. At test time on real TILDA images, no such boundary exists — the defect blends into the surrounding weave — and the model has learned the wrong feature. In classification mode, the whole image is the evidence. The model has no boundary to latch onto and must instead discriminate based on the global texture statistics that genuinely differ across the four defect classes. This is exactly the supervision that TILDA's whole-image labels provide.

B. The Broken Thread Resolution Problem

At 224×224 input resolution, a thread gap of 1–2 pixels in the original 768-pixel-wide TILDA image maps to less than one pixel in the resized input. No classifier, regardless of depth, can reliably detect a signal that has been discarded during resizing. The gap between broken thread (83.0%) and the other classes (88–92%) is therefore a resolution artifact, not a model capacity problem.

Two architectural responses are worth pursuing. Processing a 640×640 centre crop before global average pooling would preserve the full thread-gap signal at the cost of higher memory and compute. A dual-resolution branch — fusing features from both 224×224 (semantic context) and 640×640 (fine texture detail) inputs — would provide both signals simultaneously. Both are reserved for follow-on work.

C. The Calibration Class Finding

The discovery that e6 and e7 in TILDA are calibration sequences rather than defect images has not appeared in any prior publication using this dataset. Its practical importance is substantial: a model trained with e6 included would learn to associate underexposed imagery with a defect label, and a model trained with e7 would learn to associate changes in apparent fabric scale with a defect. Either corruption would degrade test accuracy in ways that are hard to diagnose without going back to the raw description files. We recommend that any future research using the full TILDA archive begin with the systematic text-file audit described in Section 5.

D. Deployment Characteristics

Running at 24 FPS on a consumer laptop CPU without a GPU makes this system accessible to any mid-scale textile mill without capital investment in machine-vision hardware. The Tkinter GUI needs no browser, web server, or internet connection, which aligns with the network isolation policies common in factory environments. The CSV export format is compatible with standard ERP quality modules and ISO 9001 traceability requirements. The confidence threshold slider allows the operator to tune the sensitivity-to-false-alarm trade-off in real time as fabric type or lighting conditions change during a production run.

E. Scope and Limitations

All results reported here come from the TILDA laboratory benchmark under controlled illumination. How the system generalises to other weave structures, yarn types, colours, or defect taxonomies has not been tested and should be validated with factory-floor data before any production deployment. The four-class taxonomy excludes important defect types found in commercial production, including warp density variation, dye-lot colour deviation, and selvage faults. Extending coverage to these classes would require additional labelled data, likely collected through an active-learning sampling strategy on live production lines.

X. Conclusion

We have presented a complete, CPU-deployable real-time fabric inspection system that closes the loop from a live camera feed to a commercially interpretable quality verdict and selling-price suggestion. The YOLOv8m-cls classification backbone, trained on 1,408 CLAHE-preprocessed TILDA images with a five-component augmentation strategy, achieves 88.5% top-1 accuracy across four defect classes and runs at 24 FPS on an Intel i5 laptop with no GPU. A novel quality scoring formula — combining per-class severity weighting with a quadratic price penalty — translates defect frequency into discount recommendations that align with commercial textile grading norms.

Three technical findings are worth highlighting beyond the accuracy numbers. First, classification mode outperforms detection mode by 63.5 percentage points on TILDA, not because it is architecturally superior but because it uses the supervision signal the dataset actually provides. Second, the broken-thread accuracy gap (83% vs. 88–92% for other classes) is a resolution artifact that a higher-resolution input branch could close without increasing model depth. Third, the systematic text-file audit

revealed two calibration classes in TILDA that would corrupt any training run that included them, a finding not previously documented in the literature.

Future directions include: (1) a dual-resolution classification branch (224×224 plus 640×640) to raise broken-thread accuracy without sacrificing inference speed; (2) expansion of the defect taxonomy through active-learning data collection on live production lines; and (3) empirical calibration of the price penalty exponent against historical transaction data from commercial textile manufacturers to replace the analytically chosen quadratic with a data-driven fit.

References

[1] H. Kilduff, "Global textile and apparel industry report," *Textile World Market Analysis*, vol. 14, no. 2, pp. 1–18, 2022.

[2] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Ind. Electron.*, vol. 55, no. 1, pp. 348–363, Jan. 2008.

[3] J. Yang, S. Li, Z. Wang, H. Dong, J. Wang, and S. Tang, "Fabric defect detection using a convolutional neural network compared with a support vector machine," *J. Text. Inst.*, vol. 111, no. 12, pp. 1834–1843, 2020.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, Las Vegas, NV, Jun. 2016, pp. 779–788.

[6] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv:2004.10934*, Apr. 2020.

[7] G. Jocher *et al.*, "YOLOv5," GitHub, 2021. [Online]. Available: `https://github.com/ultralytics/yolov5`

[8] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF CVPR*, Vancouver, 2023, pp. 7464–7475.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: `https://github.com/ultralytics/ultralytics`

[10] X. Zhang, F. Ding, Z. Luo, and G. Niu, "Fabric defect detection based on multi-scale feature extraction and attention mechanism," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6152–6160, Sep. 2021.

[11] H. Schulz-Mirbach, "Ein Referenzdatensatz zur Evaluierung von Sichtprüfungsverfahren für Textiloberflächen," Technical Report, University of Hamburg, 1995. [Online]. Available: `https://lmb.informatik.uni-freiburg.de/resources/datasets/tilda.en.html`

[12] H. Y. T. Ngan, G. K. H. Pang, and N. H. C. Yung, "Automated fabric defect detection—a review," *Image Vis. Comput.*, vol. 29, no. 7, pp. 442–458, Jun. 2011.

[13] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, Nov. 1973.

[14] S. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 11, no. 7, pp. 674–693, Jul. 1989.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, Las Vegas, NV, Jun. 2016, pp. 770–778.

[16] X. Zhang, F. Ding, Z. Luo, and G. Niu, "Fabric defect detection based on multi-scale feature extraction and attention mechanism," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6152–6160, 2021.

[17] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE CVPR*, Salt Lake City, UT, Jun. 2018, pp. 7132–7141.

[18] Z. Ren, F. Fang, N. Yan, and Y. Wu, "Automated textile defect recognition system of an industrial camera using a deep convolutional neural network," *Appl. Sci.*, vol. 12, no. 5, p. 2549, Feb. 2022.

[19] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, Montreal, Dec. 2015, pp. 91–99.

[20] F. Zhou, J. Chen, and Z. Xu, "Multi-class fabric defect detection using improved Faster R-CNN," *Pattern Recognit. Image Anal.*, vol. 33, no. 2, pp. 312–323, 2023.

[21] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Proc. ECCV*, Amsterdam, 2016, pp. 21–37.

[22] Y. Li, W. Zhang, R. Liu, and G. Wang, "Fabric defect detection based on improved YOLOv5," *Text. Res. J.*, vol. 93, no. 1–2, pp. 42–55, Jan. 2023.

[23] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE CVPR*, Honolulu, HI, Jul. 2017, pp. 2117–2125.

[24] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE CVPR*, Salt Lake City, UT, Jun. 2018, pp. 8759–8768.

[25] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI Conf. Artif. Intell.*, New York, Feb. 2020, vol. 34, pp. 12993–13000.

[26] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. ECCV*, Zurich, Sep. 2014, pp. 740–755.

[27] T. DeVries and G. W. Taylor, "Improved regularization of convolutional neural networks with Cutout," *arXiv:1708.04552*, Aug. 2017.

[28] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[29] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, San Diego, CA, May 2015.

[30] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, Toulon, France, Apr. 2017.

[31] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *Proc. IEEE/CVF CVPR*, Seattle, WA, Jun. 2020, pp. 10781–10790.

[32] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE/CVF ICCV*, Seoul, Korea, Oct. 2019, pp. 9627–9636.

[33] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Proc. ECCV*, Milan, Italy, Sep. 2024, pp. 1–18.

[34] A. Wang *et al.*, "YOLOv10: Real-time end-to-end object detection," *arXiv:2405.14458*, May 2024.

[35] Y. Zhao *et al.*, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF CVPR*, Seattle, WA, Jun. 2024, pp. 16965–16974.

[36] S. Chen, J. Liu, and W. Zhao, "Scale-adaptive transformer for woven fabric defect detection," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 2, pp. 1402–1415, Apr. 2024.